RORT

Rapport Projet

Réalisé par : Reda EL MRIBET, Thierry ZHENG, Maxime APARICIO, Ronan LE PRAT

Sommaire

1 Introduction **3**

2 Branch-and-price **4**
2.1 Règle de branchement 4
2.2 Règle de parcourt 4
2.3 Warm start 5
2.4 Obtention d'une borne inférieur du problème global 5
2.5 Règle de branchement: conflit 5

3 Heuristique primale intégrée au branch-and-price **6**
3.1 Position du problème 6
3.2 Nature exacte de l'heuristique 6
3.3 Détail technique de l'implémentation 6
3.3.1 Déclenchement de l'heuristique 6
3.3.2 Construction du pool heuristique 7
3.3.3 Résolution du maître entier restreint 7
3.4 Lien avec le mécanisme de warm start 8

4 Résultats **9**
4.1 Comparaisons de différentes méthodes sur la plus petite instance. 9
4.1.1 Solution via heuristique existante 9
4.1.2 Solution via branch-and-price simple 9
4.1.3 Solution via branch-and-price + warm-start 9
4.1.4 Solution via branch-and-price + warm-start + heuristique intégrée 10

1 Introduction

Ce projet traite un problème de routage et d'ordonnancement de trains dans une station de maintenance. On dispose d'un réseau ferroviaire représenté sous forme de graphe, d'un horizon temporel discrétisé, ainsi que d'un ensemble de contraintes propres à chaque train : instant d'arrivée, instant de départ, services obligatoires à réaliser, et incompatibilités d'occupation sur les ressources.

2 Branch-and-price

2.1 Règle de branchement

Nous avons besoin d'une règle qui influence les sous problèmes. il faut que le choix fait modifie les sous problèmes.

Contexte

Dans le programme maître, les variables de décision sont des *chemins* générés dynamiquement par le sous-problème (plus court chemin). Une solution fractionnaire au nœud courant de l'arbre Branch-and-Price se traduit par l'existence, pour un même train k, de deux chemins P_1 et P_2 avec des valeurs fractionnaires $\lambda_{P_2}, \lambda_{P_2} \in]0, 1[$.

Identification du point de divergence

Soient $P_1 = (s, v_1, v_2, \ldots, v_k, \ldots, d)$ et $P_2 = (s, v_1, v_2, \ldots, v'_k, \ldots, d)$ deux chemins fractionnaires pour le train t, partageant le même nœud source s et le même nœud destination d.

On identifie le **premier arc de divergence**, c'est-à-dire le premier arc $e_1 = (u, v)$ tel que :

$$e_1 \in P_1 \setminus P_2$$

Le nœud u est le dernier nœud commun aux deux chemins avant leur séparation. On récupère aussi e_2 l'arc appartenant à P_2 et pas à P_1

Construction des deux nœuds fils

On partitionne l'ensemble les arc sortant de u en

$$E_1 \cup E_2 = \delta^+(u),$$

En faisant bien attention à ce que la partition sépare les chemins

$$e_1 \in E_1, e_2 \in E_2$$

Dans le fils de droite, on interdit les arcs de E_1 , et dans celui de gauche, on interdit ceux de E_2 Ainsi, dans les fils on a bien des sous problèmes différents puisque le graphe sur lesquels ils sont basé sont différents

Nous avons ainsi trois méthode différentes pour creer les partitions E_1 et E_2

- E_1 ne contient que e_1 et E_2 prend le reste des arcs (Plus limitant d'abord)
- E_2 ne contient que e_2 et E_1 prend le reste des arcs (Moins limitant d'abord)
- on répartit les arêtes de manière équilibrée

Nous rappelons que nous faisons un parcours à gauche en premier.

2.2 Règle de parcourt

Nous avons deuxw règles de parcourts :

- Profondeur simple
- Profondeur puis, une fois une solutions réalisable trouvée, largeur

2.3 Warm start

La résolution de la relaxation linéaire du problème maître par génération de colonnes peut s'avérer coûteuse en temps de calcul, notamment en raison du nombre potentiellement élevé d'itérations nécessaires avant qu'aucune colonne de coût réduit négatif ne puisse plus être générée.

Afin d'accélérer les résolutions successives à chaque nœud de l'arbre de branchement, on initialise le problème maître avec un ensemble de colonnes, dit de *warm-start*, hérité du nœud parent. Il convient toutefois de veiller à exclure les colonnes devenues interdites en raison des contraintes de branchement imposées au nœud courant.

Deux stratégies ont été envisagées :

- Initialiser le nœud enfant uniquement avec les colonnes dont la variable associée λ est strictement positive dans la solution du nœud parent.
- Initialiser le nœud enfant avec l'ensemble des colonnes du nœud parent, y compris celles dont la variable λ est nulle.

La seconde stratégie a été retenue, car elle permet d'accélérer davantage la convergence. En pratique, la conservation des colonnes associées à un λ nul n'entraîne pas de surcoût.

2.4 Obtention d'une borne inférieur du problème global

La valeur optimale de la relaxation au nœud racine constitue une première borne inférieure du problème global. Cette borne peut cependant être améliorée au cours de l'exploration de l'arbre de Branch-and-Price.

En effet, à tout instant de l'algorithme, la borne inférieure globale est donnée par le minimum des bornes inférieures des nœuds encore actifs (non élagués), c'est-à-dire les nœuds appartenant à la *frontière* de l'arbre exploré.

2.5 Règle de branchement: conflit

En ouverture, nous proposons une autre règle de branchement. Dans une solution fractionnaire, il se peut que deux trains empruntent le même arc, c'est à dire, le même rail physique sur le même pas de temps. On pourrait ainsi brancher sur ce conflit. Dans l'un des noeud fils, on donne la priorité au premier train (ce qui en pratique consiste à interdire cette arc a tout les autres trains), et dans l'autre noeuds fils, on interdit au premier train de prendre cet arc.

3 Heuristique primale intégrée au branch-and-price

3.1 Position du problème

Dans l'implémentation, la relaxation linéaire est résolue par génération de colonnes. La fonction principale de décomposition est appelée via la procédure :

`dw(...)` dans `src/tusp_column_generation.cpp`. Cette fonction retourne une structure `DWResults` contenant notamment :

- `continuous_obj` : la valeur de la relaxation continue ;
- `extended_paths` : l'ensemble des colonnes générées, sous la forme de paires `(lambda, Path)`.

Le branch-and-price explore ensuite un arbre de nœuds `BandPNode`, chaque nœud contenant au minimum :

- une liste d'arcs interdits par train, `forbidden_arcs` ;
- un ensemble de colonnes réutilisées pour le warm start, `warm_paths` ;
- la profondeur `depth` ;
- la borne héritée du parent `prev_obj`.

Le problème pratique est le suivant : la relaxation fournit souvent une bonne borne, mais la solution obtenue est fractionnaire. Sans solution entière de bonne qualité, la borne supérieure reste faible, le nombre de nœuds explorés augmente, et l'arbre du branch-and-price devient coûteux à explorer.

3.2 Nature exacte de l'heuristique

L'heuristique intégrée est une **heuristique primale de type restricted master heuristic**. Elle ne génère pas de nouveaux chemins. Elle exploite au contraire les colonnes déjà produites par la décomposition de Dantzig–Wolfe et tente d'en extraire rapidement une solution entière. Plus précisément, à partir de l'ensemble des chemins contenus dans `dwresults.extended_paths`, on construit un sous-problème entier restreint, puis on résout ce sous-problème au moyen de la fonction :

```
solve_restricted_milp(...)
```

dans `src/tusp_column_generation.cpp`. L'idée est donc :

1. résoudre la relaxation continue par génération de colonnes ;
2. récupérer les colonnes courantes ;
3. résoudre un petit maître entier sur ces colonnes ;
4. mettre à jour l'incumbent si une meilleure solution entière est trouvée.

3.3 Détail technique de l'implémentation

3.3.1 Déclenchement de l'heuristique

Dans la boucle principale du branch-and-price, après l'appel à `dw(...)`, le solveur dispose de :

- la borne continue `dwresults.continuous_obj` ;
- l'ensemble des colonnes générées `dwresults.extended_paths`.

Une heuristique est ensuite déclenchée périodiquement. Dans le code actuel, cela correspond à un test du type :

```
if (nodes_explored % 10 == 1)
```

Autrement dit, l'heuristique n'est pas exécutée à chaque nœud, mais seulement à intervalles réguliers, ce qui permet de limiter son coût.

3.3.2 Construction du pool heuristique

Le pool de colonnes utilisé par l'heuristique est construit à partir de `dwresults.extended_paths`. Dans la version actuellement intégrée, on forme un vecteur `lp_paths` contenant les chemins produits par le DW :

```
for (auto [lambda, path] :  dwresults.extended_paths) lp_paths.push_back(path);
```

Le principe général est le suivant :

- les colonnes utilisées proviennent exclusivement de la relaxation courante ;
- elles sont déjà compatibles avec le nœud courant du branch-and-price, car elles ont été générées par le DW de ce nœud ;
- elles forment donc un sous-ensemble réaliste pour tenter une reconstruction entière.

Sur le plan algorithmique, il s'agit d'une heuristique fondée sur l'hypothèse suivante : les colonnes présentes dans la relaxation courante constituent une base de recherche pertinente pour produire rapidement une solution entière.

3.3.3 Résolution du maître entier restreint

Le cœur de l'heuristique est l'appel :

```
DWResults heuristic_results = solve_restricted_milp(...);
```

Cette fonction reconstruit un maître entier sur le sous-ensemble de chemins fourni. Techniquement, elle :

1. crée un modèle Gurobi ;
2. ajoute une variable binaire par chemin du pool ;
3. impose les contraintes de sélection d'un chemin par train ;
4. impose les contraintes de couverture des services ;
5. impose les contraintes de conflits sur les arcs ;
6. minimise le coût total.

On retrouve donc la même logique que dans le maître final, mais appliquée à un ensemble de colonnes beaucoup plus petit.

Si la solution entière trouvée a une valeur strictement inférieure à `best_objective`, alors :

- `best_objective` est mis à jour ;
- `best_paths` est remplacé ;
- `best_extended_paths` est remplacé par la solution heuristique.

Ainsi, l'heuristique met directement à jour l'*incumbent* global du branch-and-price.

3.4 Lien avec le mécanisme de warm start

L'intégration de l'heuristique s'inscrit dans une logique plus large de la reutilisation de colonnes entre les noeuds de l'arbre. Après séparation, deux noeuds fils sont créés. Pour chacun d'eux, le solveur construit un ensemble `warm_paths` à partir des colonnes du noeud parent, en supprimant les chemins incompatibles avec les nouveaux arcs interdits. Cela peut se faire, par exemple, via une fonction locale de filtrage du type :

```
auto filter_paths = [](vector<vector<int>>& forbidden) -> vector<Path>
```

Cette fonction parcourt les chemins du parent et supprime ceux qui utilise un arc devenu interdit pour le train branché. L'heuristique et le warm start sont donc complémentaires :

- le `warm_start` sert à accélérer la résolution DW des nœuds enfants.
- l'heuristique sert à extraire plus vite une solution entière à partir des colonnes disponibles.

4 Résultats

4.1 Comparaisons de différentes méthodes sur la plus petite instance.

Nous rappelons les trois méthodes de branchements utilisé :

1. Plus limitant d'abord, on interdit le plus d'arcs
2. Moins limitant d'abord, on interdit le moins d'arcs
3. on répartit les arêtes de manière équilibrée

4.1.1 Solution via heuristique existante

La méthode existante renvoie une solution avec 2 trains rejetés.

Instance	Méthode	Valeur	Borne inf	Écart (%)	Nombre de noeuds	Temps heur. (s)
1	Heuristique MILP seule	5322	1959.5	171.60%	1	1.6

Table 1: Résultats obtenus avec l'heuristique pré-existante sur la plus petite instance.

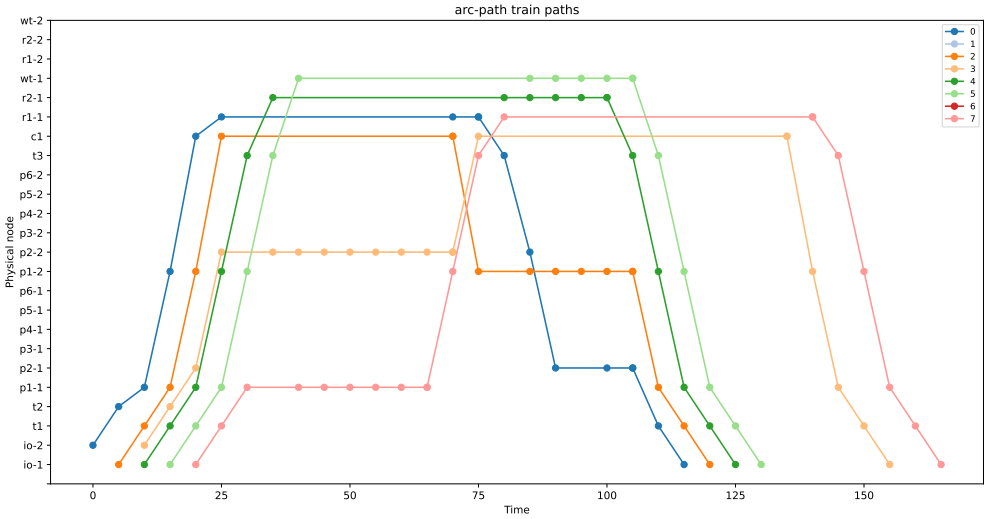

Figure 1: Répartition des trains par solution heuristique pré-existante.

4.1.2 Solution via branch-and-price simple

On voit qu'on arrive à trouver une meilleure solution, qui ne rejette plus qu'un seul train, au détriment d'un temps de calcul bien allongé.

Instance	Méthode	Valeur	Borne inf	Écart (%)	Nombre de noeuds	Temps heur. (s)
1	Branch-and-price	3646	1959.5	89.07%	1	600

Table 2: Résultats obtenus avec l'heuristique pré-existante sur la plus petite instance.

4.1.3 Solution via branch-and-price + warm-start

Toutes les méthodes s'exécutent plus rapidement. La seconde méthode n'arrive pas à trouver de solution, même après avoir exploré 300 noeuds en profondeur. La première méthode est celle qui renvoie la meilleure solution, qui ne rejette qu'un seul train.

ID	Heuristique	Heur. branch.	Solution	Borne inf	Gap (%)	Nb noeuds	Temps (s)
1	No	1	3641	1959.5	85.81%	300	104.5
2	No	2	N/A	1959.5	N/A	300	68
3	No	3	3660	1959.5	86.78%	300	89.8

Table 3: Comparaison des stratégies de branchement avec warm-start.

On peut aussi constater que l'heuristique 2 s'exécute plus rapidement que les deux autres, ce qui s'explique par le fait qu'en descendant en profondeur on doit générer moins de colonnes au fur et à mesure des noeuds explorés, contrairement aux deux autres méthodes qui elles reviennent à des noeuds de profondeur moindre.

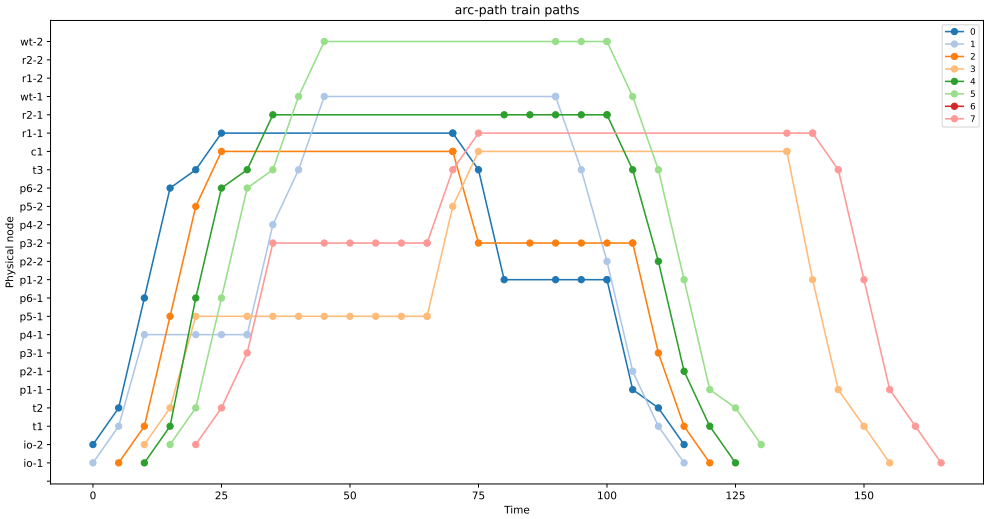

Figure 2: Meilleure solution obtenue avec stratégie de branchement 1.

4.1.4 Solution via branch-and-price + warm-start + heuristique intégrée

On inclut cette fois l'heuristique dans la résolution que l'on ré-exécute périodiquement afin d'essayer de trouver de nouvelles bonnes solutions entières. L'ensemble des trois méthodes renvoie de meilleures solutions globalement. La deuxième méthode réussit à renvoyer une solution de très bonne qualité acceptant tous les trains.

ID	Heuristique	Heur. branch.	Solution	Borne inf	Gap (%)	Nb noeuds	Temps (s)
4	Yes	1	3641	1959.5	85.81%	300	97.4
5	Yes	2	1965	1959.5	0.28%	300	93.9
6	Yes	3	3641	1959.5	85.81%	300	97.9

Table 4: Comparaison des stratégies de branchement avec heuristique périodiquement activée.

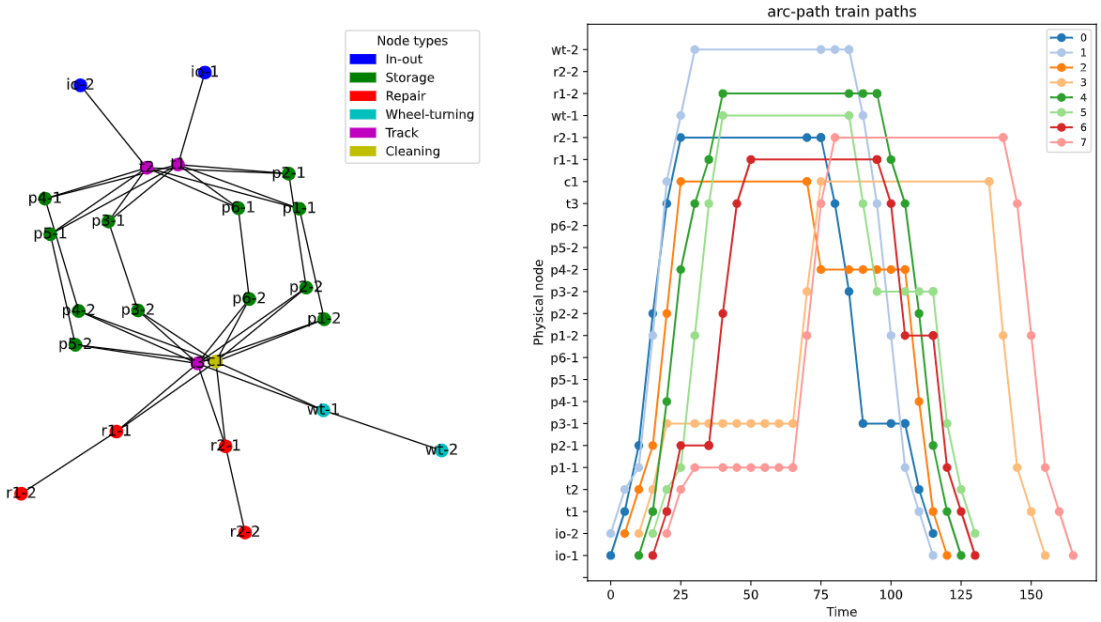

Figure 3: Meilleure solution obtenue avec stratégie de branchement 1.

Annexe

Données brutes

Table 5: Comparaison des méthodes pour small station – scénario 2

Méthode	**Valeur retournée**	**Temps d'exécution (s)**	**Gap à l'opt (%)**
BP	Aucune solution trouvée	249.10	–
BP + sep2	Aucune solution trouvée	246.15	–
BP + sep3	Aucune solution trouvée	212.33	–
BP + heuristique	17391	280.74	68.68

Table 6: Comparaison des méthodes pour medium station – scénario 2

Méthode	**Valeur retournée**	**Temps d'exécution (s)**	**Gap à l'opt (%)**
BP	Aucune solution trouvée	373.13	–
BP + sep2	Aucune solution trouvée	511.79	–
BP + sep3	Aucune solution trouvée	509.73	–
BP + heuristique	40799	587.43	67.27

Table 7: Comparaison des méthodes pour large station – scénario 1

Méthode	**Valeur retournée**	**Temps d'exécution (s)**	**Gap à l'opt (%)**
BP	40239	63.31	0.00
BP + sep2	40239	62.13	0.00
BP + sep3	40239	60.52	0.00
BP + heuristique	40239	64.16	0.00

Table 8: Comparaison des méthodes pour large station – scénario 2

Méthode	**Valeur retournée**	**Temps d'exécution (s)**	**Gap à l'opt (%)**
BP	Aucune solution trouvée	85.57	–
BP + sep2	Aucune solution trouvée	656.51	–
BP + sep3	Aucune solution trouvée	78.28	–
BP + heuristique	137662	64.18	0.00

Table 9: Comparaison des méthodes pour small station – scénario 1

Méthode	**Valeur retournée**	**Temps d'exécution (s)**	**Gap à l'opt (%)**
BP	Aucune solution trouvée	4.01	–
BP + sep2	Aucune solution trouvée	3.82	–
BP + sep3	Aucune solution trouvée	3.82	–
BP + heuristique	5322	3.76	0.00